

Exam: Time Series Visualization with Bokeh

This exam tests your ability to visualize time series data using the Bokeh library. You will be working with the "Daily Minimum Temperatures in Melbourne" dataset. For each question, provide the Python code using Bokeh to generate the requested visualization.

Dataset: "daily-minimum-temperatures-in-melbourne.csv"

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("daily-minimum-temperatures-in-melbourne.csv")

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])
```

```
In [4]: # Importation des bibliothèques nécessaires
import pandas as pd
import numpy as np
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
    DateRangeSlider,
    CustomJS,
)
from bokeh.layouts import row, column, gridplot
from bokeh.transform import factor_cmap
from bokeh.palettes import Spectral11

# Activation de Bokeh dans Le notebook
output_notebook()

# Chargement du fichier CSV
donnees_brutes = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Renommage des colonnes
donnees_brutes.columns = ['Date', 'Temperature']

# Suppression des valeurs manquantes marquées '?'
donnees_brutes['Temperature'] = donnees_brutes['Temperature'].astype(str).str.replace('?', '', regex=False)
donnees_brutes['Temperature'] = pd.to_numeric(donnees_brutes['Temperature'], errors='coerce')
donnees_brutes.dropna(subset=['Temperature'], inplace=True)

# Conversion de la colonne Date en format datetime
donnees_brutes['Date'] = pd.to_datetime(donnees_brutes['Date'])

donnees_brutes.head()
```



Loading BokehJS ...

Out[4]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Question 1: Basic Time Series Line Plot

1. Create a basic line plot showing the daily minimum temperature over time.

- Use the 'Date' column on the x-axis and the 'Temperature' column on the y-axis.
- Set the plot title to "Daily Minimum Temperatures".
- Label the x-axis as "Date" and the y-axis as "Temperature (°C)".
- Add tooltips to display the date and temperature when hovering over the line.
- Enable pan, wheel zoom, and reset tools.

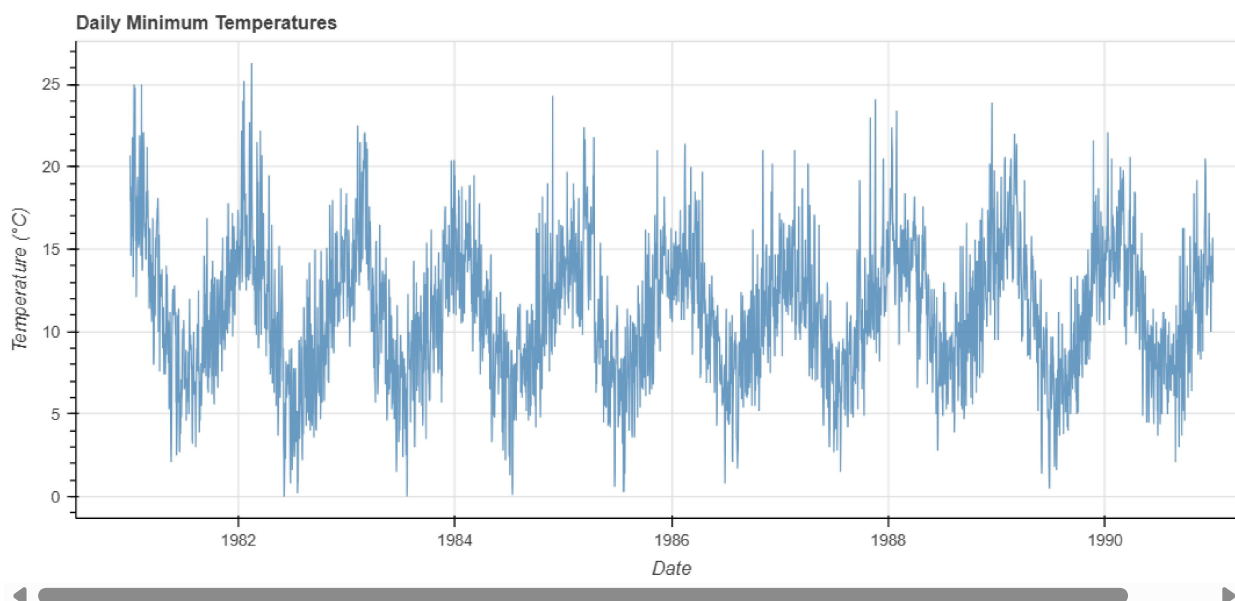
```
In [5]: # Source de données Bokeh
source_q1 = ColumnDataSource(donnees_brutes)

# Création du graphique Linéaire
graphique_temp = figure(
    title="Daily Minimum Temperatures",
    x_axis_label="Date",
    y_axis_label="Temperature (°C)",
    x_axis_type="datetime",
    width=900,
    height=400,
    tools="pan,wheel_zoom,reset",
)

# Tracé de La Ligne
ligne_temp = graphique_temp.line(
    x='Date', y='Temperature', source=source_q1,
    line_width=1, color='steelblue', alpha=0.8,
)

# Tooltip au survol
info_bulle_q1 = HoverTool(
    renderers=[ligne_temp],
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"},
    mode='vline',
)
graphique_temp.add_tools(info_bulle_q1)
graphique_temp.xaxis.formatter = DatetimeTickFormatter(months="%b %Y", years="%Y")

show(graphique_temp)
```



Question 2: Rolling Average 2. Calculate the 30-day rolling average of the daily minimum temperature and plot it alongside the original temperature data.

- * Create a new column 'Rolling_Avg' in the DataFrame containing the 30-day rolling average.
- * Plot both the original 'Temperature' and the 'Rolling_Avg' on the same plot.
- * Use different colors and line styles to distinguish between the two.
- * Add a legend to the plot to label the lines.
- * Add tooltips to display the date, original temperature, and rolling average.

```
In [6]: # Answer 2: (Provide code here)
# Calcul de la moyenne mobile sur 30 jours
donnees_brutes['Rolling_Avg'] = donnees_brutes['Temperature'].rolling(window=30).mean()

source_q2 = ColumnDataSource(donnees_brutes)

# Graphique avec deux lignes
graphique_moyenne = figure(
    title="Températures minimales quotidiennes avec moyenne mobile sur 30 jours",
    x_axis_label="Date", y_axis_label="Temperature (°C)",
    x_axis_type="datetime", width=900, height=400,
    tools="pan,wheel_zoom,reset",
)

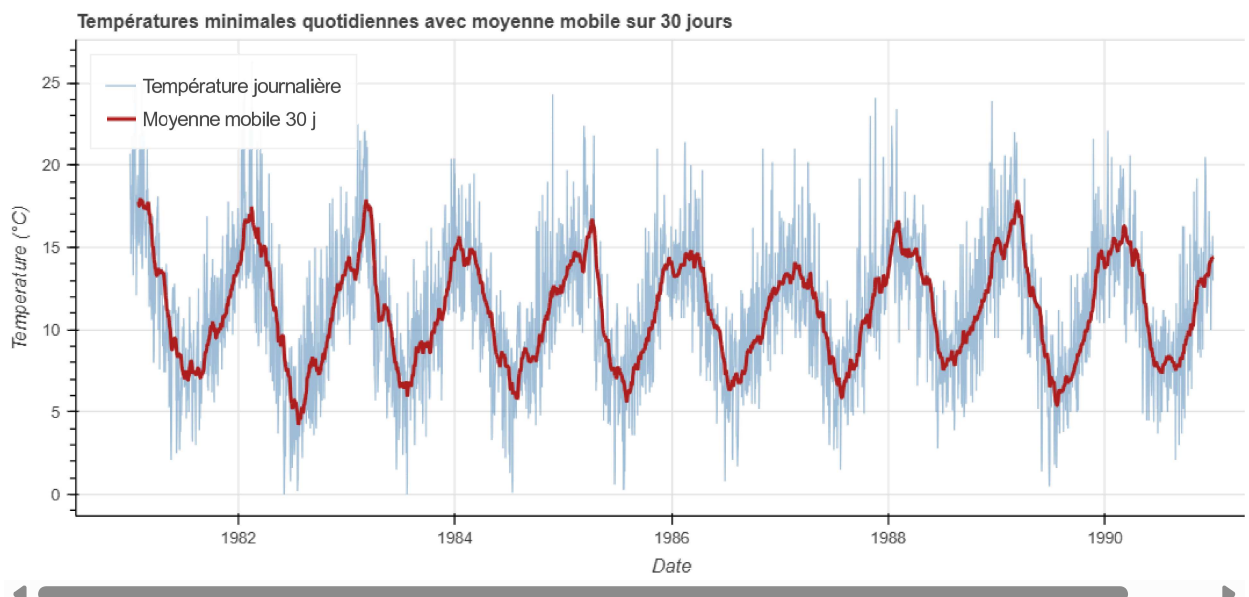
# Ligne originale (bleue, semi-transparente)
ligne_originale = graphique_moyenne.line(
    x='Date', y='Temperature', source=source_q2,
    line_width=1, color='steelblue', alpha=0.5,
    legend_label='Température journalière',
)

# Ligne moyenne mobile (rouge)
ligne_moyenne = graphique_moyenne.line(
    x='Date', y='Rolling_Avg', source=source_q2,
    line_width=2.5, color='firebrick',
    legend_label='Moyenne mobile 30 j',
)

# Tooltip
info_bulle_q2 = HoverTool(
    renderers=[ligne_originale],
    tooltips=[
        ("Date", "@Date{%F}"),
        ("Température", "@Temperature{0.0} °C"),
        ("Moyenne mobile", "@Rolling_Avg{0.00} °C"),
    ],
    formatters={"@Date": "datetime"}, mode='vline',
)

graphique_moyenne.add_tools(info_bulle_q2)
graphique_moyenne.xaxis.formatter = DatetimeTickFormatter(months="%b %Y", years="%Y")
graphique_moyenne.legend.location = "top_left"
graphique_moyenne.legend.click_policy = "hide"

show(graphique_moyenne)
```



Question 3: Monthly Box Plots 3. Create box plots to visualize the distribution of temperatures for each month.

- * Extract the month from the 'Date' column and create a new 'Month' column.
- * Group the data by 'Month' and prepare it for plotting.
- * Use Bokeh's box plot elements to visualize the distribution.
- * Label the x-axis with month names and the y-axis with "Temperature (°C)".
- * Add tooltips to display the month and relevant statistical values (min, max, media

In [7]: # Answer 3: (Provide code here)

```
import calendar

# Extraction du mois et du nom du mois
donnees_brutes['Mois_Num'] = donnees_brutes['Date'].dt.month
donnees_brutes['Nom_Mois'] = donnees_brutes['Date'].dt.month.apply(lambda x: calendar.month_abbr[x])
ordre_mois = [calendar.month_abbr[i] for i in range(1, 13)]

# Calcul des statistiques descriptives par mois
groupe_mois = donnees_brutes.groupby('Nom_Mois')['Temperature']
stats_mois = pd.DataFrame({
    'q1': groupe_mois.quantile(0.25),
    'q2': groupe_mois.quantile(0.50),
    'q3': groupe_mois.quantile(0.75),
    'mini': groupe_mois.min(),
    'maxi': groupe_mois.max(),
}).reindex(ordre_mois)

# Calcul des moustaches (règle 1.5 × IQR)
stats_mois['ecart_inter'] = stats_mois['q3'] - stats_mois['q1']
stats_mois['moustache_bas'] = (stats_mois['q1'] - 1.5 * stats_mois['ecart_inter']).clip(lower=stats_mois['mini'])
stats_mois['moustache_haut'] = (stats_mois['q3'] + 1.5 * stats_mois['ecart_inter']).clip(upper=stats_mois['maxi'])
stats_mois['mois'] = stats_mois.index

source_q3 = ColumnDataSource(stats_mois)
largeur_boite = 0.6

# Graphique
graphique_mois = figure(
    title="Distribution mensuelle des températures",
    x_range=ordre_mois, x_axis_label="Month", y_axis_label="Temperature (°C)",
    width=900, height=450, tools="pan,wheel_zoom,reset",
)

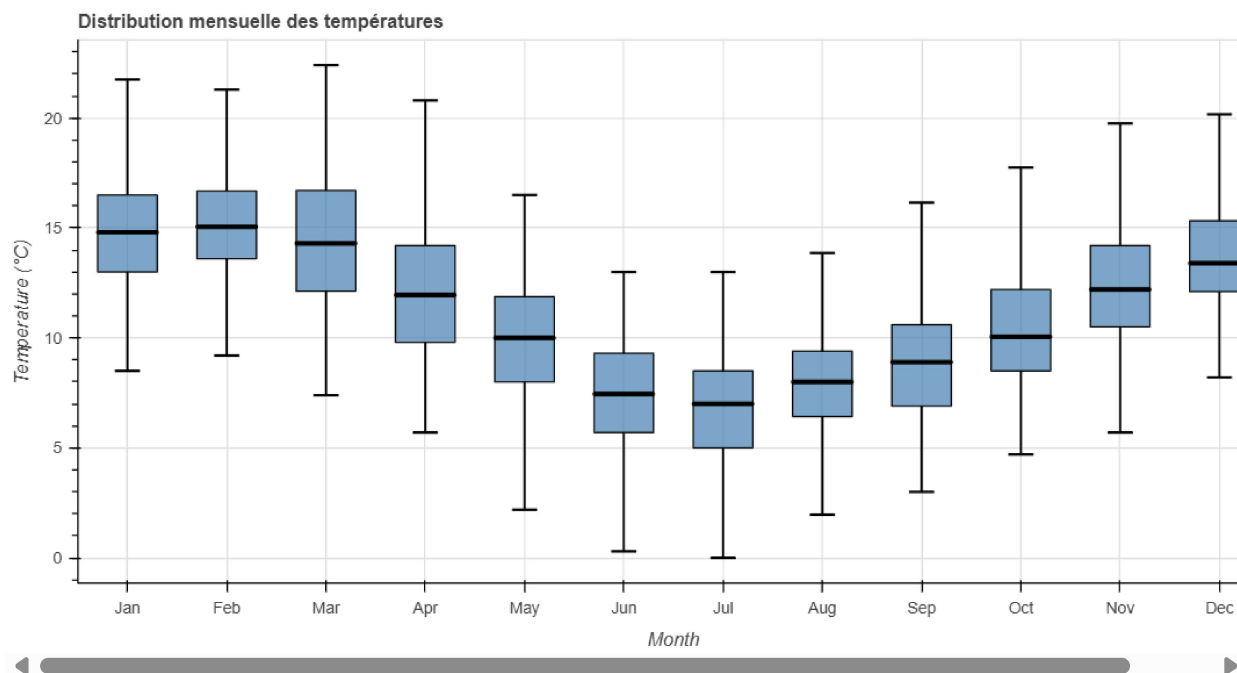
# Tiges et extrémités des moustaches
graphique_mois.segment('mois', 'moustache_haut', 'mois', 'q3', source=source_q3, line_color='black', line_width=1.5)
graphique_mois.segment('mois', 'moustache_bas', 'mois', 'q1', source=source_q3, line_color='black', line_width=1.5)
graphique_mois.rect('mois', 'moustache_haut', largeur_boite*0.4, 0.05, source=source_q3, fill_color='black', line_color='black')
graphique_mois.rect('mois', 'moustache_bas', largeur_boite*0.4, 0.05, source=source_q3, fill_color='black', line_color='black')

# Boîtes IQR
boites_mois = graphique_mois.vbar(
    x='mois', width=largeur_boite, bottom='q1', top='q3',
    source=source_q3, fill_color='steelblue', fill_alpha=0.7, line_color='black',
)

# Ligne médiane
graphique_mois.rect('mois', 'q2', largeur_boite, 0.08, source=source_q3, fill_color='white', line_color='black', line_color='black')

# Tooltip
graphique_mois.add_tools(HoverTool(renderers=[boites_mois], tooltips=[
    ("Mois", "@mois"), ("Max", "@maxi{0.0} °C"), ("Q3", "@q3{0.0} °C"),
    ("Médiane", "@q2{0.0} °C"), ("Q1", "@q1{0.0} °C"), ("Min", "@mini{0.0} °C"),
]))

show(graphique_mois)
```



```
In [8]: from bokeh.plotting import output_notebook, show
import pandas as pd

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Now you can proceed with your Bokeh plotting code!
print(df.head()) # Just to see if the dataframe loaded correctly

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

df
```



Loading BokehJS ...

	Date	DailyTemperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Out[8]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8
...	...	...
3645	1990-12-27	14.0
3646	1990-12-28	13.6
3647	1990-12-29	13.5
3648	1990-12-30	15.7
3649	1990-12-31	13.0

3650 rows × 2 columns

4. Create box plots to visualize the distribution of temperatures for each year, and use color mapping to highlight temperature variations.

- Extract the year from the 'Date' column and create a new 'Year' column.
- Group the data by 'Year' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution for each year.
- Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
- Use `factor_cmap` to color the boxes based on the median temperature of each year.
- Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
- Enable pan, wheel zoom, and reset tools.

In [9]:

```
# Answer 4: (Provide code here)
# Extraction de l'année
donnees_brutes['Annee'] = donnees_brutes['Date'].dt.year.astype(str)
liste_annees = sorted(donnees_brutes['Annee'].unique().tolist())

# Statistiques par année
groupe_annee = donnees_brutes.groupby('Annee')['Temperature']
stats_annee = pd.DataFrame({
    'q1': groupe_annee.quantile(0.25),
    'q2': groupe_annee.quantile(0.50),
    'q3': groupe_annee.quantile(0.75),
    'mini': groupe_annee.min(),
    'maxi': groupe_annee.max(),
    'moy': groupe_annee.mean(),
}).reindex(liste_annees)

# Moustaches
stats_annee['ecart_inter'] = stats_annee['q3'] - stats_annee['q1']
stats_annee['moustache_bas'] = (stats_annee['q1'] - 1.5 * stats_annee['ecart_inter']).clip(lower=stats_annee['mini'])
stats_annee['moustache_haut'] = (stats_annee['q3'] + 1.5 * stats_annee['ecart_inter']).clip(upper=stats_annee['maxi'])
stats_annee['annee'] = stats_annee.index

source_q4 = ColumnDataSource(stats_annee)

# Palette de couleurs par année
palette_annees = factor_cmap(field_name='annee', palette=Spectral11, factors=liste_annees)

# Graphique
graphique_annee = figure(
    title="Distribution annuelle des températures",
    x_range=liste_annees, x_axis_label="Year", y_axis_label="Temperature (°C)",
    width=900, height=450, tools="pan,wheel_zoom,reset",
)

largeur_annee = 0.6

# Tiges et caps
graphique_annee.segment('annee', 'moustache_haut', 'annee', 'q3', source=source_q4, line_color='black', line_width=1.
graphique_annee.segment('annee', 'moustache_bas', 'annee', 'q1', source=source_q4, line_color='black', line_width=1.
graphique_annee.rect('annee', 'moustache_haut', largeur_annee*0.4, 0.05, source=source_q4, fill_color='black', line_c
graphique_annee.rect('annee', 'moustache_bas', largeur_annee*0.4, 0.05, source=source_q4, fill_color='black', line_c

# Boîtes colorées
```

```

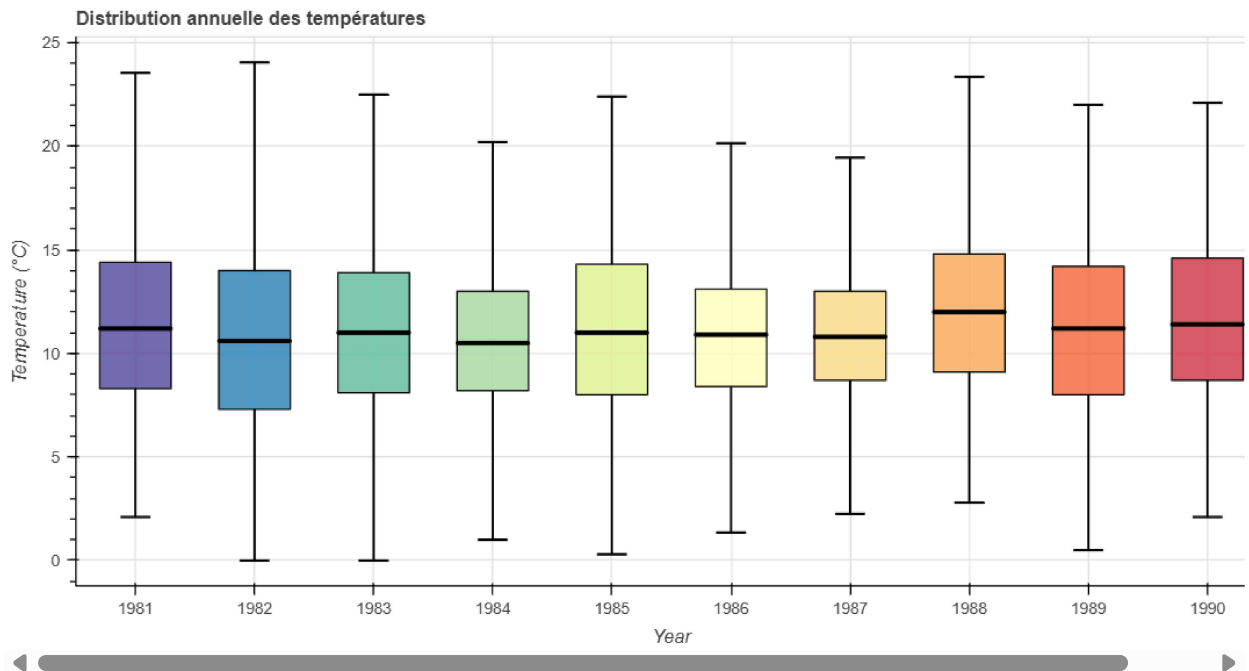
boites_annee = graphique_annee.vbar(
    x='annee', width=largeur_annee, bottom='q1', top='q3',
    source=source_q4, fill_color=palette_annees, fill_alpha=0.85, line_color='black',
)

# Médiane
graphique_annee.rect('annee', 'q2', largeur_annee, 0.08, source=source_q4, fill_color='white', line_color='black', li

# Tooltip
graphique_annee.add_tools(HoverTool(renderers=[boites_annee], tooltips=[
    ("Année", "@annee"), ("Max", "@maxi{0.0} °C"), ("Q3", "@q3{0.0} °C"),
    ("Médiane", "@q2{0.0} °C"), ("Moyenne", "@moy{0.00} °C"),
    ("Q1", "@q1{0.0} °C"), ("Min", "@mini{0.0} °C"),
]))

show(graphique_annee)

```



Question 5: Interactive Time Range Selection

5. Create an interactive line plot where the user can select a specific time range to view using a date range slider.

- Create a basic line plot of 'Temperature' over 'Date'.
- Implement a date range slider using Bokeh widgets to allow users to select a start and end date.
- Update the plot dynamically based on the selected date range.
- Add tooltips to display the date and temperature.
- Enable pan, wheel zoom, and reset tools.

```

In [10]: # Answer 5: (Provide code here)
# Deux sources : complète et filtrée
source_compleete = ColumnDataSource(donnees_brutes)
source_filtree = ColumnDataSource(donnees_brutes)

# Graphique interactif
graphique_interactif = figure(
    title="Températures - Sélection interactive de période",
    x_axis_label="Date", y_axis_label="Temperature (°C)",
    x_axis_type="datetime", width=900, height=400,
    tools="pan,wheel_zoom,reset",
)

ligne_interactive = graphique_interactif.line(
    x='Date', y='Temperature', source=source_filtree,
    line_width=1.5, color='steelblue', alpha=0.8,
)

# Tooltip
graphique_interactif.add_tools(HoverTool(
    renderers=[ligne_interactive],
    tooltips=[("Date", "@Date{%F}"), ("Température", "@Temperature{0.0} °C")],
    formatters={"@Date": "datetime"}, mode='vline',
))
graphique_interactif.xaxis.formatter = DatetimeTickFormatter(months="%b %Y", years="%Y")

```



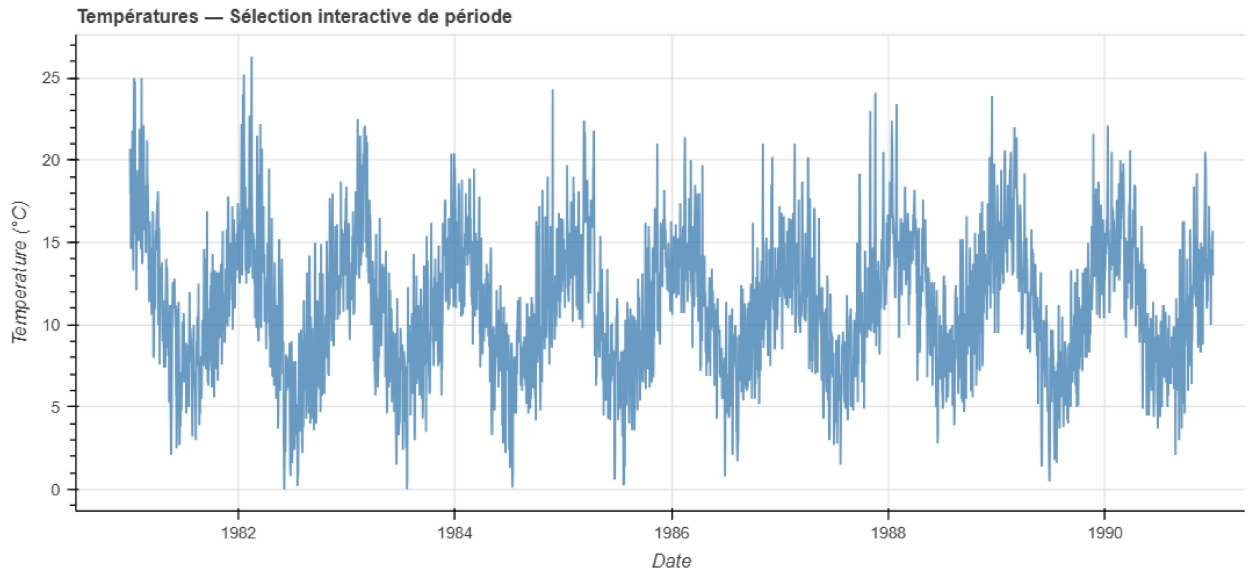
```

# Curseur de dates
casseur_dates = DateRangeSlider(
    title="Sélectionner une période",
    start=donnees_brutes['Date'].min(),
    end=donnees_brutes['Date'].max(),
    value=(donnees_brutes['Date'].min(), donnees_brutes['Date'].max()),
    step=1, width=880,
)

# Callback JavaScript pour filtrer dynamiquement
callback_filtre = CustomJS(
    args=dict(source_complete=source_complete, source_filtrée=source_filtrée, curseur=casseur_dates),
    code="""
    const donnees_completes = source_complete.data;
    const donnees_filtrees = {Date: [], Temperature: []};
    const [debut, fin] = curseur.value;
    for (let i = 0; i < donnees_completes['Date'].length; i++) {
        const d = donnees_completes['Date'][i];
        if (d >= debut && d <= fin) {
            donnees_filtrees['Date'].push(d);
            donnees_filtrees['Temperature'].push(donnees_completes['Temperature'][i]);
        }
    }
    source_filtrée.data = donnees_filtrees;
    source_filtrée.change.emit();
    """
)
casseur_dates.js_on_change('value', callback_filtre)

mise_en_page = column(graphique_interactif, curseur_dates)
show(mise_en_page)

```



Question 6: Time Series Decomposition Visualization

6. Perform a simple time series decomposition to visualize the trend and seasonality components of the temperature data.

- Resample the data to monthly frequency and calculate the monthly average temperature.
- Use a simple moving average to estimate the trend component.
- Calculate the seasonal component by subtracting the trend from the original monthly data.
- Create three separate Bokeh plots: one for the original monthly data, one for the trend, and one for the seasonal component.
- Ensure the plots are aligned and share the same x-axis (Date).
- Add tooltips to each plot to display the date and corresponding value.
- Enable pan, wheel zoom, and reset tools for each plot.

```

In [11]: import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,

```



```

        NumeralTickFormatter,
    )
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

df

```



BokehJS 3.8.2 successfully loaded.

Out[11]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8
...	...	...
3645	1990-12-27	14.0
3646	1990-12-28	13.6
3647	1990-12-29	13.5
3648	1990-12-30	15.7
3649	1990-12-31	13.0

3650 rows × 2 columns

```

In [12]: # Rééchantillonnage à fréquence mensuelle
donnees_mensuelles = donnees_brutes.set_index('Date').resample('MS')['Temperature'].mean().reset_index()
donnees_mensuelles.columns = ['Date', 'Temp_Mensuelle']

# Tendance : moyenne mobile sur 12 mois
donnees_mensuelles['Tendance'] = donnees_mensuelles['Temp_Mensuelle'].rolling(window=12, center=True).mean()

# Saisonnalité : écart entre les données mensuelles et la tendance
donnees_mensuelles['Saisonnalité'] = donnees_mensuelles['Temp_Mensuelle'] - donnees_mensuelles['Tendance']

# Suppression des NaN pour les graphiques tendance et saisonnalité
donnees_propres = donnees_mensuelles.dropna()

source_mensuel = ColumnDataSource(donnees_mensuelles)
source_tendance = ColumnDataSource(donnees_propres)
source_saison = ColumnDataSource(donnees_propres)

formateur_date = DatetimeTickFormatter(months="%b %Y", years="%Y")
params_communs = dict(x_axis_type="datetime", width=900, height=250, tools="pan,wheel_zoom,reset")

# Graphique 1 : données mensuelles
graph_mensuel = figure(title="Température mensuelle moyenne",
                        x_axis_label="Date", y_axis_label="Temp (°C)", **params_communs)
l1 = graph_mensuel.line('Date', 'Temp_Mensuelle', source=source_mensuel, line_width=2, color='steelblue')
graph_mensuel.add_tools(HoverTool(renderers=[l1], formatters={"@Date": "datetime"},
                                tooltips=[("Date", "@Date{%b %Y}"), ("Temp", "@Temp_Mensuelle{0.00} °C")]))
graph_mensuel.xaxis.formatter = formateur_date

# Graphique 2 : tendance (axe x partagé)
graph_tendance = figure(title="Tendance (moyenne mobile 12 mois)",
                        x_axis_label="Date", y_axis_label="Temp (°C)",
                        x_range=graph_mensuel.x_range, **params_communs)
l2 = graph_tendance.line('Date', 'Tendance', source=source_tendance, line_width=2, color='firebrick')
graph_tendance.add_tools(HoverTool(renderers=[l2], formatters={"@Date": "datetime"},

```

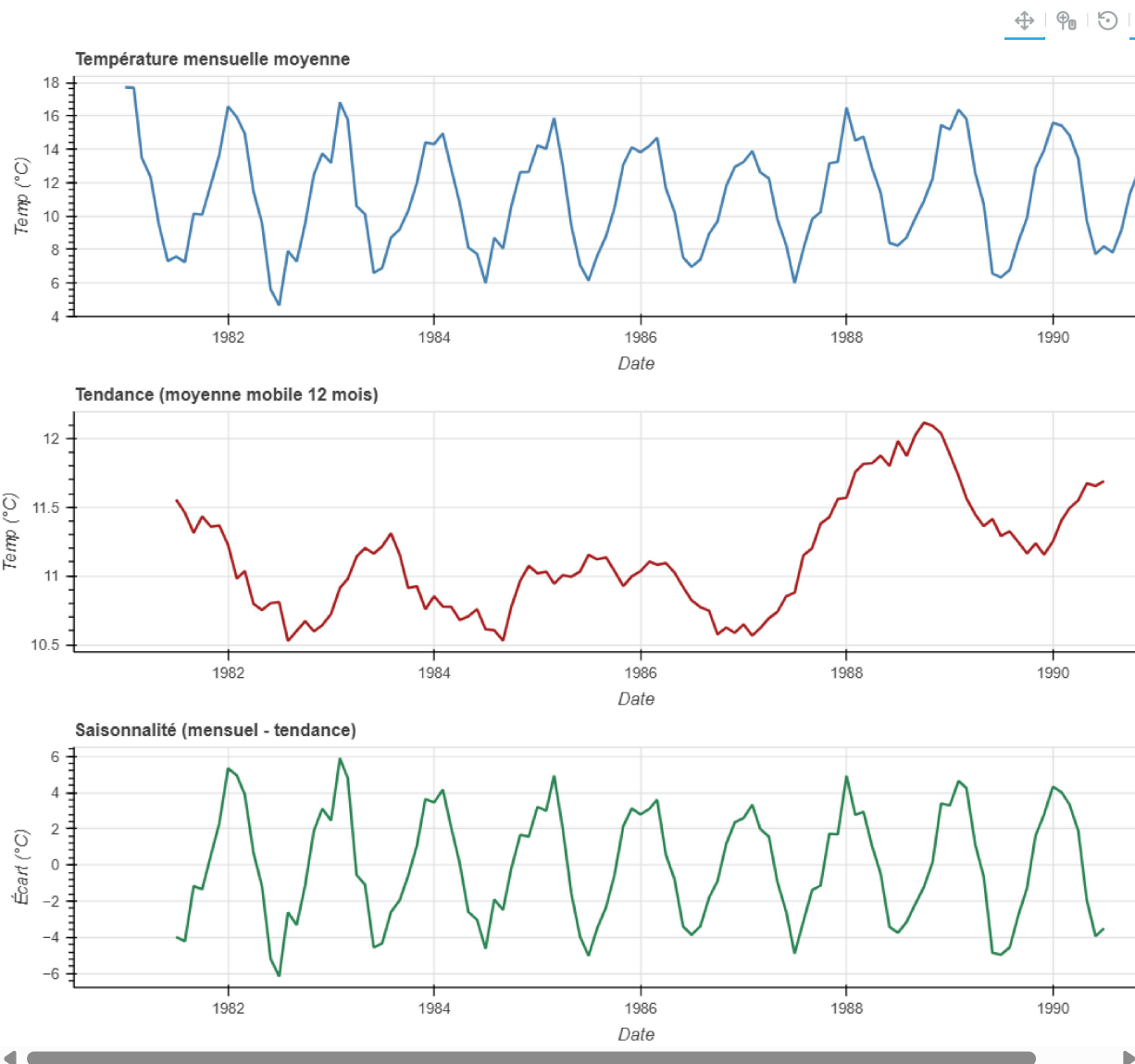
```

    tooltips=[("Date", "@Date{%b %Y}"), ("Tendance", "@Tendance{0.00} °C)")]
graph_tendance.xaxis.formatter = formateur_date

# Graphique 3 : saisonnalité (axe x partagé)
graph_saison = figure(title="Saisonnalité (mensuel - tendance)",
                      x_axis_label="Date", y_axis_label="Écart (°C)",
                      x_range=graph_mensuel.x_range, **params_communs)
l3 = graph_saison.line('Date', 'Saisonnalite', source=source_saison, line_width=2, color='seagreen')
graph_saison.add_tools(HoverTool(renderers=[l3], formatters={"@Date": "datetime"},
                                tooltips=[("Date", "@Date{%b %Y}"), ("Saisonnalité", "@Saisonnalite{0.00} °C)"])))
graph_saison.xaxis.formatter = formateur_date

# Affichage des 3 graphiques alignés verticalement avec axe x partagé
disposition_finale = gridplot([[graph_mensuel], [graph_tendance], [graph_saison]])
show(disposition_finale)

```



In []: